



ELECTRÓNICA DIGITAL III

CLASE 4

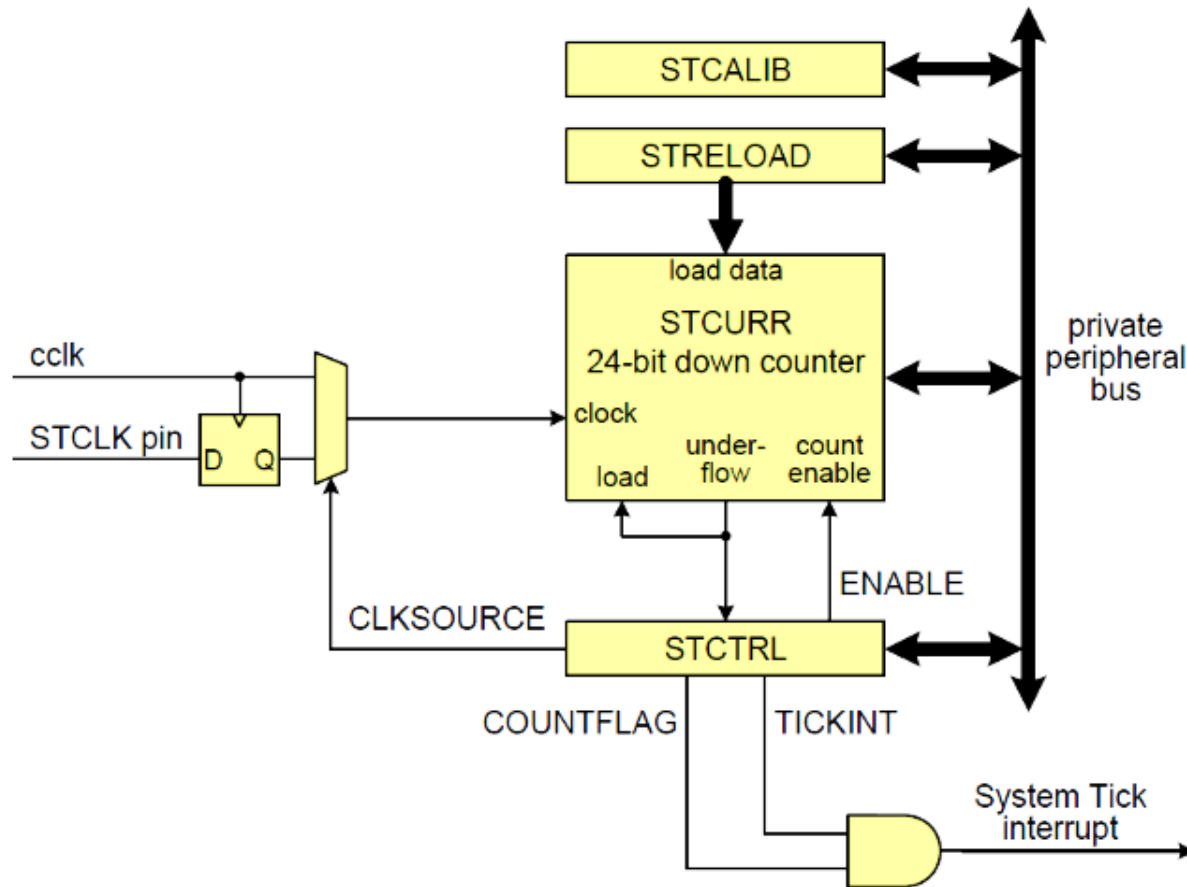
Módulo Interno SysTick

Ing. Emiliano Migliore

Módulo SysTick

El SysTick (System Tick Timer) es un temporizador dedicado al sistema que se ubica en el core del CPU y está diseñado por ARM. Por tanto, no es un periférico.

Dado que el SysTick se comunica con el core a través del NVIC, su funcionamiento se basa en la generación de pedidos de interrupción cada cierto intervalo de tiempo fijo.



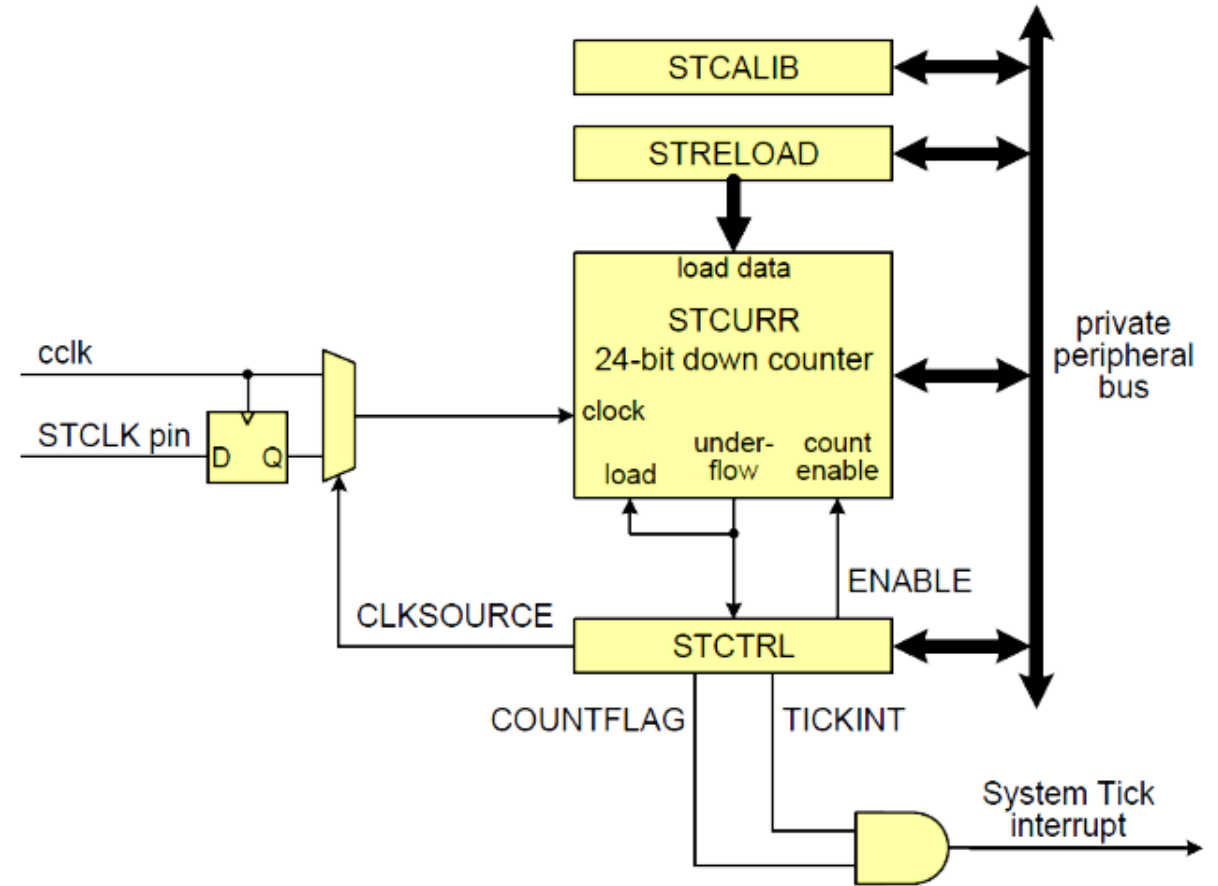
System Tick Timer block diagram

Módulo SysTick

Inicialmente, el registro STCALIB (System Timer Calibration value register) contiene el valor por defecto ante reset (0xF423F), que se cargará, a partir del código booteo, en el registro STRELOAD (System Timer Reload value register).

Una vez que el registro STCTRL (System Timer Control and Status register) identifica cual será la fuente de reloj a la cual se decrementará el contador de 24bits a través del registro STCURRE (System Timer Current value register) y habilita el contador (Count Enable), este decrementa el valor en STRELOAD hasta llegar a cero produciendo un pedido de interrupción, y una nueva orden de carga de STCURRE con el valor de STRELOAD.

La fuente de reloj, puede ser interna (CCLK – reloj del CPU) o externa (STCLK – P3.26).



System Tick Timer block diagram

Módulo SysTick

El tiempo de desbordamiento del SysTick t_{stick} dependerá del valor del registro RELOAD n_{stick} y de la frecuencia de la fuente de reloj f_{CLK} . Por tanto, este se determina como:

$$\frac{1}{f_{CLK}} [s] \rightarrow 1[cuentas]$$
$$t_{stick} [s] \rightarrow n_{stick} + 1[cuentas]$$

Entonces:

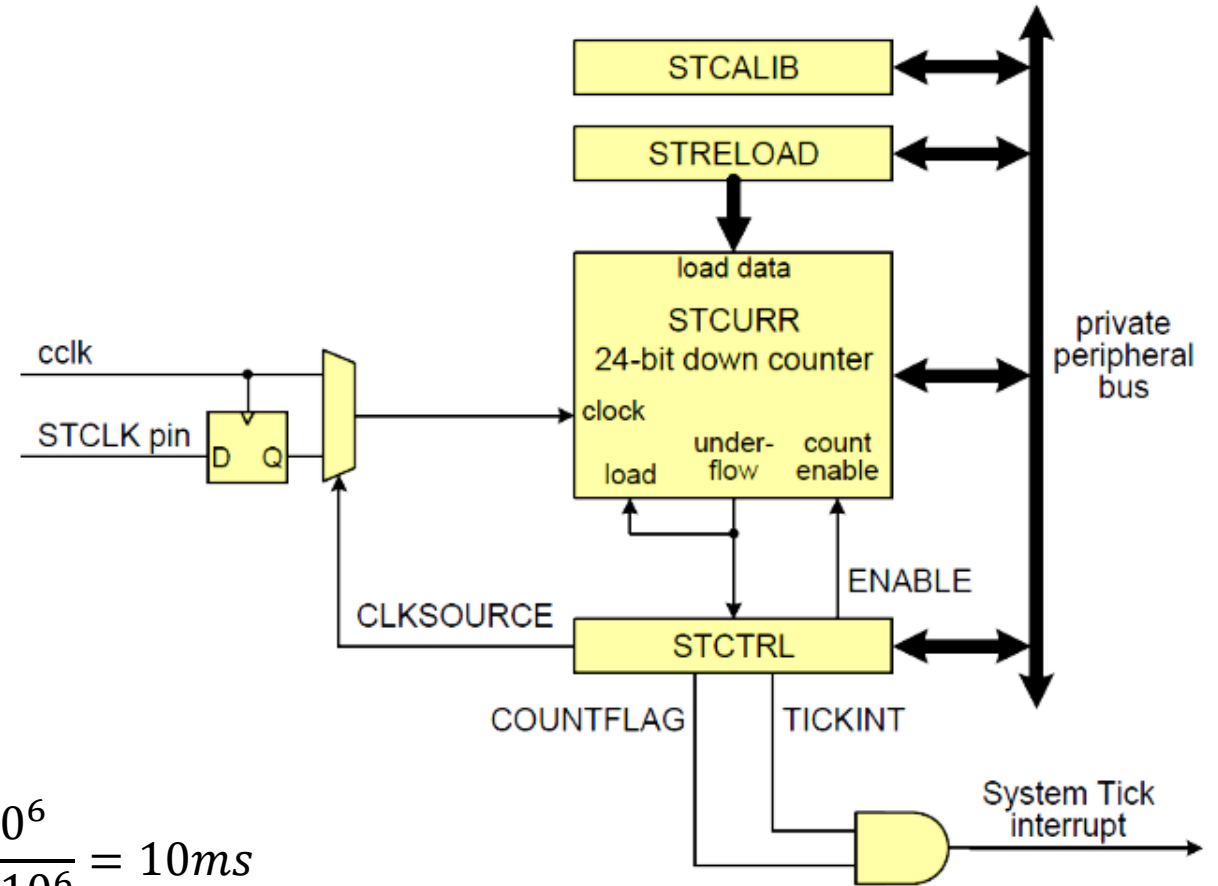
$$t_{stick} = \frac{n_{stick} + 1}{f_{CLK}} \Rightarrow n_{stick} = f_{CLK} t_{stick} - 1$$

Puesto que se trata de un contador decreciente de 24 bits, y para una frecuencia de Clock de 100MHz resulta el siguiente tiempo por defecto:

$$t_{stick} = \frac{n_{stick} + 1}{f_{CLK}} = \frac{0XF423F + 1}{100MHz} = \frac{999999 + 1}{100 \times 10^6} = \frac{1 \times 10^6}{100 \times 10^6} = 10ms$$

Luego, el tiempo máximo es:

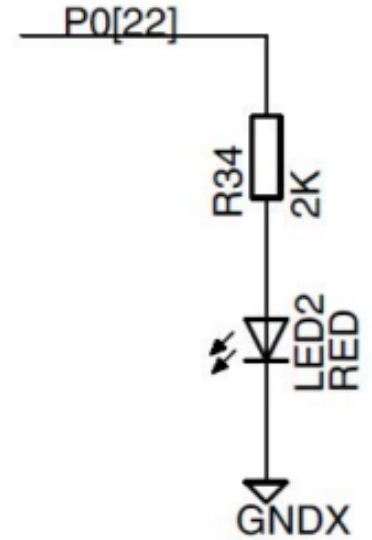
$$t_{stick} = \frac{n_{stick} + 1}{f_{CLK}} = \frac{0xFFFFFFFF + 1}{100MHz} = 167,77ms$$



System Tick Timer block diagram

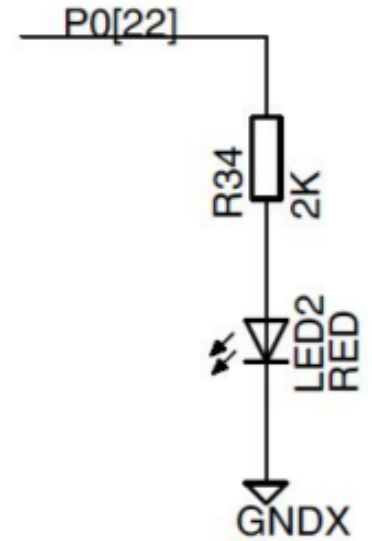
Ejemplo 1: Interrupción por SysTick

```
1  /*
2   * @brief  Enciende y apaga un led conectado al pin P0.22 de forma intermitente
3             cada 100ms debido al desborde del SysTick
4   */
5
6  /* Directivas de Inclusión */
7  #include "LPC17xx.h"
8
9  /* Definición de Variables Globales */
10 uint32_t nTicks = 0x98967F;    //Desborde cada 100ms a 100MHz
11 uint32_t vTicks = 0;
12
13 /* Prototipos de Funciones */
14 void cfgGPIO(void);
15 void cfgSysTick(void);
16
17 /* Función Principal */
18 int main(void) {
19
20     cfgGPIO();
21     cfgSysTick();
22
23     while(1) {}
24
25     return 0;
26 }
27
28 /* Definiciones de Funciones */
29 void cfgGPIO(void)
30 {
31     LPC_PINCON->PINSEL1 &= ~(0b11<<12); // P0.22 GPIO Function
32     LPC_GPIO0->FIODIR  |= (1<<22);      // P0.22 Digital Output
33     return;
34 }
35
```



Ejemplo 1: Interrupción por SysTick

```
36 void cfgSysTick(void)
37 {
38     SysTick->LOAD = nTicks;    //Carga cuentas en STRELOAD
39     SysTick->VAL = 0;           //Carga cuentas a cero STCURR
40     SysTick->CTRL = 0x07;       /* Habilita Contador (ENABLE) (1<<0)
41                                * Habilita Interrupcion (TICKINT) (1<<1)
42                                * Selecciona fuente de reloj (CLKSOURCE) (1<<2)
43                                */
44     return;
45 }
46
47 void SysTick_Handler(void)
48 {
49     vTicks++;
50     if(vTicks%2){
51         LPC_GPIO0->FIOSET = (1<<22);
52     }
53     else{
54         LPC_GPIO0->FIOCLR = (1<<22);
55     }
56     SysTick->CTRL &= SysTick->CTRL; //Limpia Flag SysTick (COUNTFLAG)
57     return;
58 }
59
60
```



Ejercicio 1: Interrupción por SysTick y Externa

- 1.- Configure el SysTick Timer de modo que genere una forma de onda llamada PWM. Esta señal debe ser extraída por el pin P0.22 para que controle la intensidad de brillo del led. El periodo de la señal debe ser de 10ms con un duty cycle de 10%. Configure la interrupción externa EINT0 de modo que cada vez que se entre en una rutina de interrupción externa el duty cycle incremente en un 10% (1ms). Esto se repite hasta llegar al 100%, luego, si se entra nuevamente a la interrupción externa, el duty cycle volverá al 10%.
- 2.- Modificar los niveles de prioridad para que la interrupción por systick tenga mayor prioridad que la interrupción externa.

Anexo I: Modulación por Ancho de Pulso (PWM)

Una de las técnicas de control utilizada para la transferencia de potencia en circuito de corriente continua consiste en denominado control por modulación de ancho de pulso (PWM), en el que un interruptor destinado a controlar una carga, realiza la conexión/desconexión de la misma con la fuente de DC mediante la variación del ciclo de trabajo D de una señal digital periódica. Este ciclo de trabajo se define como el tiempo de estado alto t_H en relación al periodo T de la señal:

$$D = \frac{t_H}{T}$$

Como se observa en la figura 3.12, el objetivo es poder obtener una señal analógica continua promediada en el tiempo a partir de una señal digital ON/OFF, mediante la generación de pulsos positivos de frecuencia de conmutación f constante, pero en el que el tiempo de disparo de dichos pulsos varíe mediante el tiempo de estado alto de los mismos.

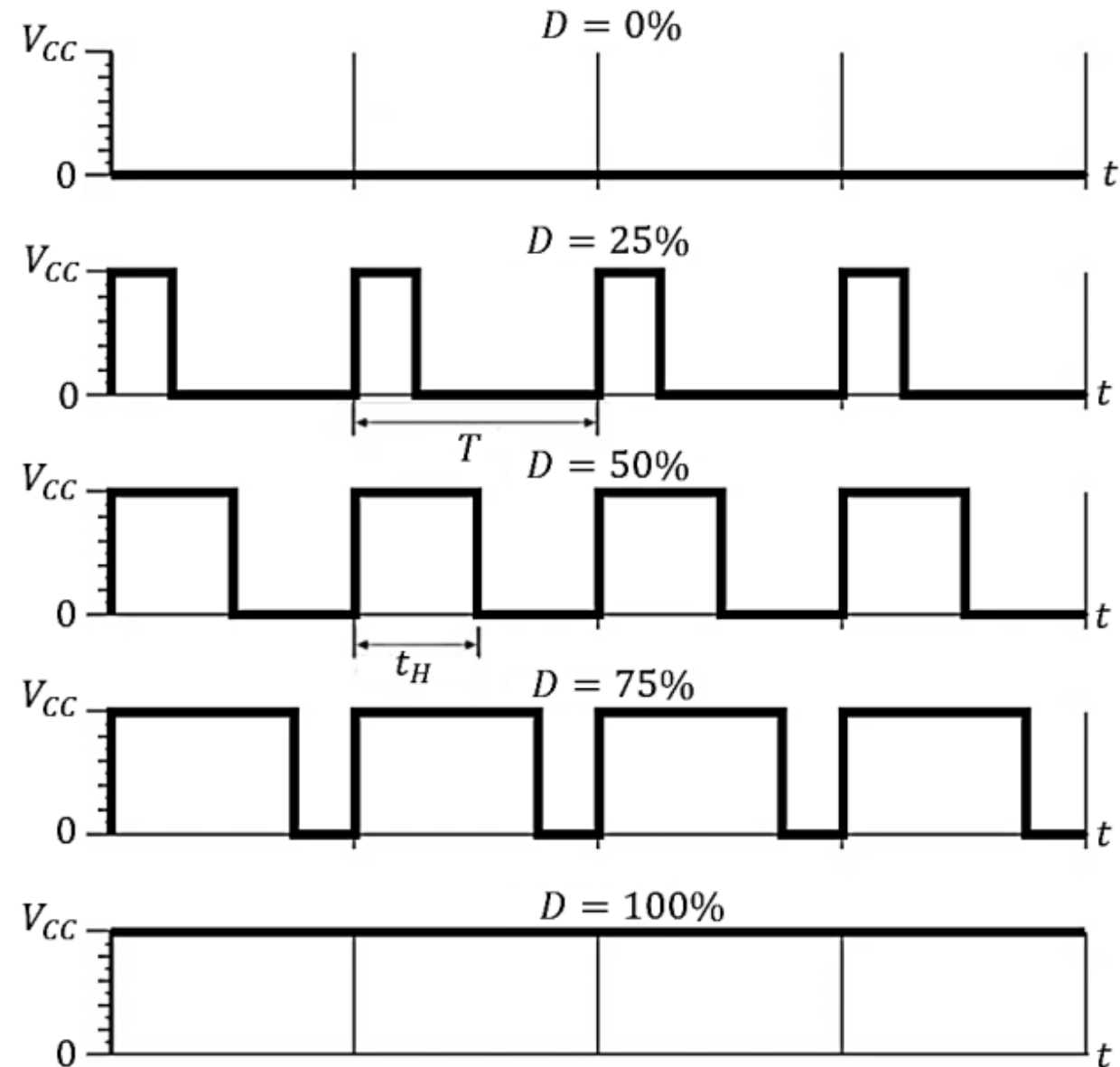


Figura 3.12: Variaciones del ciclo de trabajo en el control PWM (Fuente: Propia)